Wildcat Collateral Contracts Audit

Côme du Crest

2026-02-01

Table of contents

- Table of contents
- Wildcat Collateral Contracts Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Info] Transient storage not completely cleaned in factory
  - Optimisations and miscellaneous

Wildcat Collateral Contracts Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes selected contracts within `wildcat-finance/collateral-contract` as of commit `0x3d9f8a2`.

The contracts within scope are:

- src/interfaces/CollateralStruct.sol
- src/interfaces/IWildcatMarket.sol
- src/lens/CollateralContractData.sol
- src/lens/CollateralLens.sol
- src/SimpleMarketCollateralMultiParty.sol
- src/WildcatMarketCollateralFactory.sol

Context

The goal of the contracts is to allow wildcat market owners to deposit collateral for their markets. Approved executors will be able to liquidate the collateral for underlying market tokens when the market is delinquent.

The liquidation process swaps the collateral tokens for the underlying market tokens through authorized exchanges. The swap call data is provided by the liquidator. The collateral contract only ensures that the received amount of underlying tokens after the swap is not greater `maxRepaymentBips = 10_500 bps` times the market delinquent debt.

Collateral tokens can be reclaimed only if the market is closed and markets can be closed only if the debt has been repaid. This ensures the collateral cannot be redeemed when needed to repay market lenders.

The authorized liquidator is trusted not to profit from the trade opportunity created by the liquidation. I.e. liquidator is trusted not to provide inaccurately priced swap data and sandwich the swap transaction for its own profit.

Since the swap call data is arbitrary and controlled by the authorized liquidator, collusion in between the liquidator and the authorized exchange would result in all collateral being at risk of being stolen.

These two security assumptions could be avoided by using a price oracle for the collateral/underlying token swap. Additionally, the exchange integration could be avoided by relying on the liquidator to provide the underlying token itself. I do not think this is required if a trusted liquidator is present and using an oracle may be undesired.

Status

The report has been sent to the core developer.

No issues have been found, only minor comments have been raised.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Wildcat Protocol (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Info] Transient storage not completely cleaned in factory

Summary

The collateral contract factory uses transient storage to store parameters used by the deployed contract. The deploy function shows intent to clean transient storage after creating a contract but only clears one word out of three being used.

Vulnerability Detail

The function `WildcatMarketCollateralFactory.deployCollateralContract()` stores three words of encoded parameters to transient storage to be read by the newly created contract before attempting to empty them with `setEmpty()`:

```
    function deployCollateralContract(
        address marketAddress,
        address collateralToken
    ) public returns (address collateralContract) {
        ...

        TmpCollateralParameterStorage
            memory tmp = TmpCollateralParameterStorage({
                borrower: IWildcatMarket(marketAddress).borrower(),
                collateralToken: collateralToken,
                associatedMarket: marketAddress
            });

        _setTmpCollateralParameters(tmp);
        collateralContract = LibStoredInitCode.create2WithStoredInitCode(
            collateralInitCodeStorage,
            salt
        );
        _tmpCollateralParameters.setEmpty();

        ...
    }
```

The function `setEmpty()` only clears one word of transient storage:

```
  function setEmpty(TransientBytesArray transientSlot) internal {
    assembly {
      tstore(transientSlot, 0)
    }
  }
```

Impact

The transient storage of `WildcatMarketCollateralFactory` is not completely cleared in between different calls of `deployCollateralContract()`. While it is good practice to clean transient storage at the end of externally callable functions, it is not strictly necessary and I do not believe this poses any issue.

Code Snippets

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/src/WildcatMarketCollateralFactory.sol#L357

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/src/types/TransientBytesArray.sol#L117

Recommendation

Remove the call to `setEmpty()` if transient storage does not need to be cleaned. Otherwise, call `setEmpty()` on each written to slots to make sure the whole storage is clean.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

maxRepaymentBips could be a constant

The immutable argument `maxRepaymentBips` is set at constructor time to a fixed constant `10_500`. `maxRepaymentBips` could be turned into a constant to save gas.

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/src/SimpleMarketCollateralMultiParty.sol#L86

totalShares being uint224 likely wastes gas

The storage variable `totalShares` is `uint224` which makes the EVM use additional operations when reading and storing the value than if it were a full word. I figure the reason it is a `uint224` is to match the `uint224` shares value held by a depositor that is packed into a single word in storage with `uint32 lastFullLiquidationIndex` in the `Depositor` struct. However, `totalShares` does not strictly need to be a `uint224` and is most often used as a `uint256` in the code.

Incorrect inline documentation about full liquidation

The inline documentation of `SimpleMarketCollateralMultiParty` states:

> Any depositors with a `lastFullLiquidationIndex` less than or equal to `fullLiquidationIndex` will have their shares reset to 0.

Actually only depositors with an index strictly less than `fullLiquidationIndex` will have their shares rest to 0. This behaviour is correct but the comment is incorrect.

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/src/SimpleMarketCollateralMultiParty.sol#L44

_tmpCollateralParameters is not in storage

The `AUDIT.md` document lists `_tmpCollateralParameters` as being the first storage variable of `WildcatMarketCollateralFactory` but it is a constant and does not use storage.

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/src/WildcatMarketCollateralFactory.sol#L21

https://github.com/wildcat-finance/collateral-contract/blob/3d9f8a2c561c62f10dcddf0c49cb227896992a08/AUDIT.md?plain=1#L341